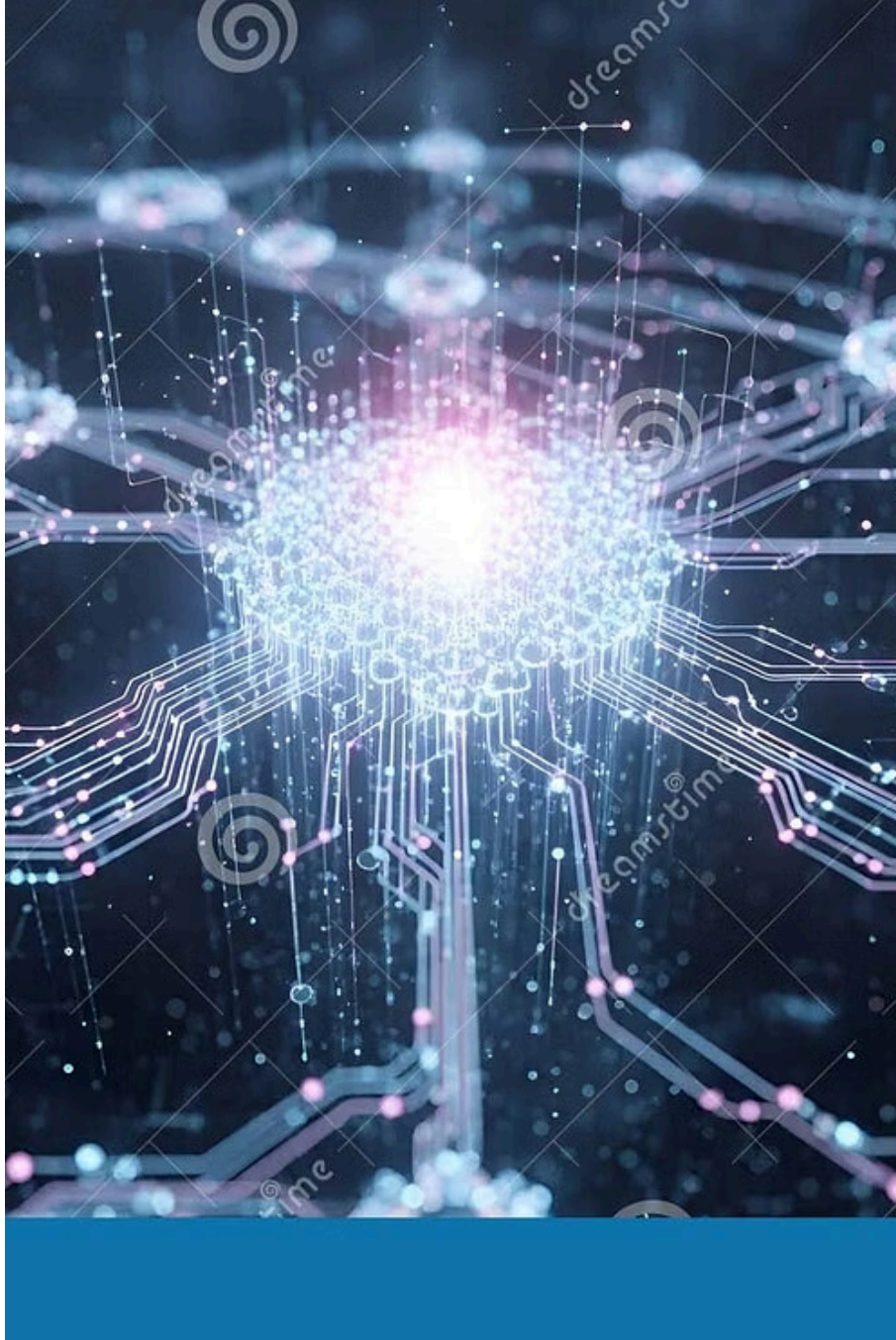
Introduction to Machine Learning

A comprehensive overview of ML types, classification, regression, clustering, generalization, overfitting, underfitting, and the K-Nearest Neighbors algorithm — from foundational concepts to hands-on implementation.

WEEK 12 · LECTURES 21–22



Types of Machine Learning

Machine learning algorithms are broadly categorized based on how they learn from data and what kind of feedback they receive. Understanding these categories is essential for selecting the right approach to a given problem.

Supervised Learning

The model is trained on labeled data — each training example has an input and a known correct output. The algorithm learns to map inputs to outputs. Common tasks include classification (e.g., spam detection) and regression (e.g., house price prediction).

Unsupervised Learning

The model is trained on data without labels. It must discover hidden patterns or structure on its own. Common tasks include clustering (e.g., customer segmentation) and dimensionality reduction (e.g., PCA for visualization).

Reinforcement Learning

An agent learns by interacting with an environment and receiving rewards or penalties. The goal is to maximize cumulative reward over time. Common applications include game playing (e.g., AlphaGo) and robotic control systems.

Classification: Binary & Multi-Class

Classification is a supervised learning task where the goal is to predict a discrete class label for a given input. It is one of the most widely used ML techniques, applied in areas ranging from medical diagnosis to fraud detection.

Binary Classification

The output variable has exactly **two possible classes**. The model learns to separate data into one of two categories.

- Spam vs. Not Spam (email filtering)
- Fraudulent vs. Legitimate (transaction monitoring)
- Disease Present vs. Absent (medical diagnosis)
- Pass vs. Fail (student assessment)

Common algorithms: Logistic Regression, Decision Trees, SVM, KNN.

Multi-Class Classification

The output variable has **more than two classes**. The model assigns each input to one of several possible categories.

- Digit recognition (0–9, ten classes)
- Species identification (e.g., Iris: Setosa, Versicolor, Virginica)
- Sentiment analysis (Positive, Neutral, Negative)
- Image classification (cats, dogs, birds, etc.)

Approaches include One-vs-Rest, One-vs-One, and native multi-class algorithms like Decision Trees and Neural Networks.

What Is Regression?

Regression is a supervised learning technique used to predict a **continuous numerical value** based on input features. Unlike classification, which outputs discrete categories, regression outputs a real number that can take any value within a range.

Linear Regression

Models the relationship between input variables and a continuous output using a straight line (or hyperplane in higher dimensions). The goal is to minimize the sum of squared errors between predicted and actual values. Simple and interpretable, it serves as a baseline for many regression problems.

Polynomial Regression

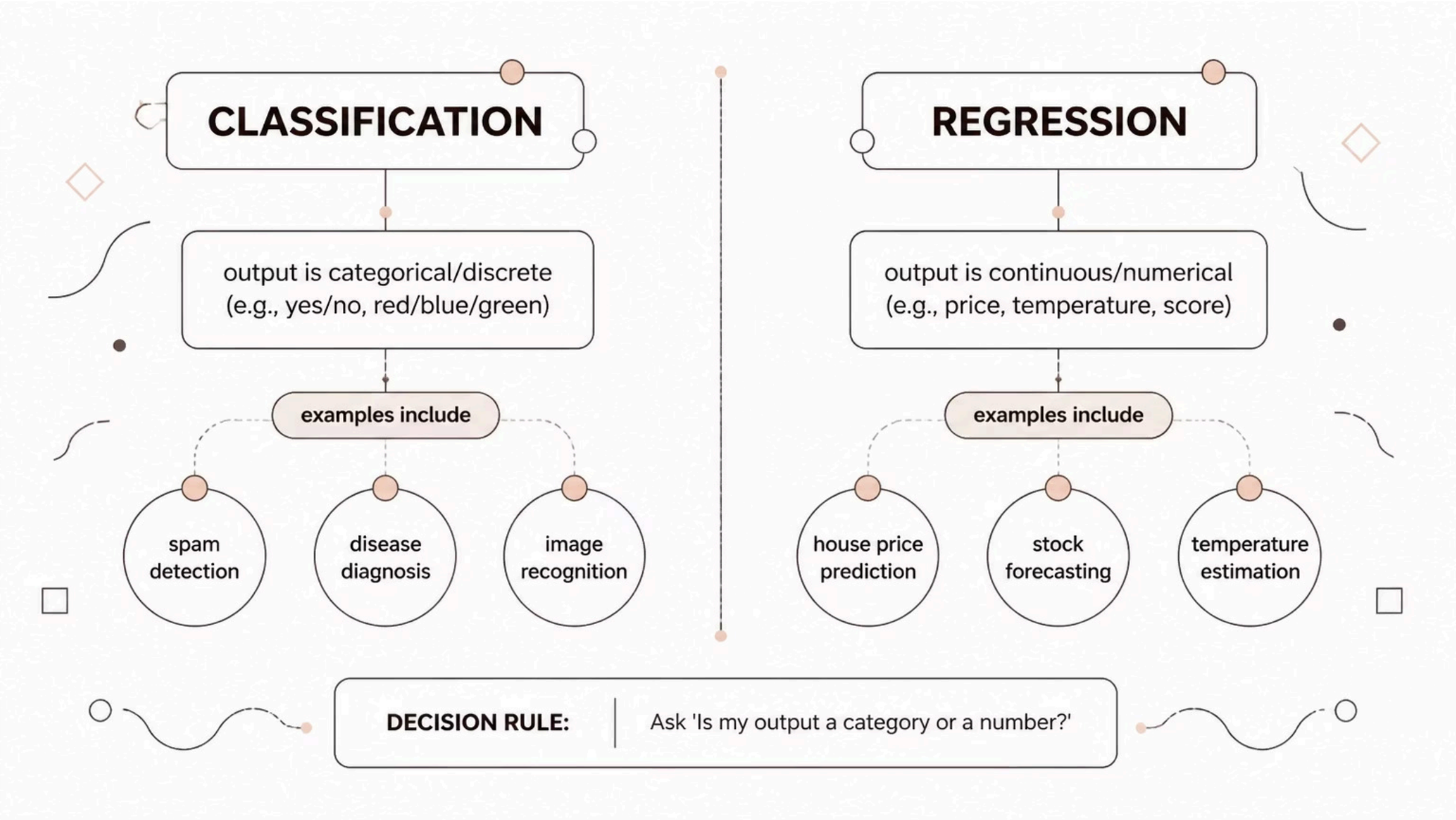
Extends linear regression by fitting a polynomial curve to the data, allowing the model to capture non-linear relationships. While more flexible, it risks overfitting if the polynomial degree is too high relative to the data size.

Real-World Applications

Regression is used to predict house prices, stock values, temperature, sales revenue, patient recovery time, and any outcome that is naturally continuous. The quality of a regression model is typically measured using metrics like Mean Squared Error (MSE) or R-squared.

Classification vs. Regression: When to Use Which?

Choosing between classification and regression depends entirely on the **nature of the target variable** — what you are trying to predict. This decision is one of the first and most critical steps in designing a machine learning solution.



Key questions to ask: **Is the output a label or category?** → Use Classification. **Is the output a measurable quantity?** → Use Regression. In some edge cases, a continuous output can be discretized into bins (e.g., age groups), converting a regression problem into classification — but this involves a trade-off in information loss.

Clustering with K-Means

K-Means is an **unsupervised learning** algorithm used to group similar data points into clusters without any pre-defined labels. The algorithm partitions data into **K clusters**, where K is specified by the user, by minimizing the within-cluster variance.

01

Initialize K Centroids

Randomly select K points in the feature space to serve as the initial cluster centers (centroids). Different initializations can lead to different final clusters, so the algorithm is often run multiple times.

02

Assign Points to Nearest Centroid

Each data point is assigned to the cluster whose centroid is closest, typically measured using Euclidean distance. This step creates K groups of points.

03

Update Centroids

Recalculate the position of each centroid as the mean (average) of all points assigned to that cluster. This moves centroids toward the center of their assigned points.

04

Repeat Until Convergence

Steps 2 and 3 are repeated until centroids no longer move significantly or a maximum number of iterations is reached. The algorithm converges to a local minimum of the within-cluster sum of squares.

Generalization, Overfitting & Underfitting

A machine learning model's ultimate goal is to **generalize** — to perform well on new, unseen data, not just the data it was trained on. Understanding the balance between model complexity and data size is central to achieving good generalization.

Overfitting

The model learns the training data **too well**, including its noise and outliers, resulting in poor performance on new data. Symptoms: very high training accuracy but low test accuracy.

Causes: Model is too complex, too many parameters, insufficient training data.

Solutions: Regularization, cross-validation, more training data, simpler model, early stopping.

Underfitting

The model is **too simple** to capture the underlying patterns in the data. It performs poorly on both training and test data. Symptoms: low accuracy on both training and test sets.

Causes: Model is too simple, insufficient features, excessive regularization.

Solutions: Increase model complexity, add more features, reduce regularization, train longer.

Model Complexity vs. Data Size: As data size increases, a more complex model can be justified without overfitting. With small datasets, simpler models generalize better. The ideal model sits at the "sweet spot" between underfitting and overfitting — the bias-variance tradeoff.

What Is K-Nearest Neighbors (KNN)?

KNN is a **supervised, instance-based (lazy) learning** algorithm used for both classification and regression. Unlike eager learners that build a model during training, KNN stores all training data and makes predictions only at query time — hence "lazy."

No Explicit Training

KNN does not construct a model during the training phase. It simply memorizes the entire training dataset. All computation is deferred until a prediction is needed, making training instant but prediction slower.

Distance-Based

Predictions are made by finding the **K closest training examples** to the query point, using a distance metric (typically Euclidean distance). The class or value of these neighbors determines the output.

Hyperparameter K

K is the number of neighbors considered. A small K (e.g., K=1) makes the model sensitive to noise (overfitting). A large K smooths decision boundaries but may oversimplify (underfitting). K is typically chosen via cross-validation.

Voting Mechanism

For **classification**: the majority class among K neighbors is assigned. For **regression**: the average (or weighted average) of the K neighbors' values is returned. Distance-weighted voting gives closer neighbors more influence.

How KNN Works: Step-by-Step with Numerical Examples

Understanding KNN through concrete numerical examples clarifies how distance calculations and voting produce predictions. Below is a worked example for both classification and the role of K.

1

Store Training Data

Suppose we have a dataset of flowers with two features: petal length and petal width, labeled as Species A or Species B. All points are stored without any model fitting.

2

Choose K and Distance Metric

Select K (e.g., K=3). Choose Euclidean distance: $d = \sqrt{[(x_2-x_1)^2 + (y_2-y_1)^2]}$. For a new flower with petal length=4.5, width=1.5, compute distance to every training point.

3

Find K Nearest Neighbors

Sort all distances and select the 3 smallest. Example distances: Point 1 (Species A) = 0.5, Point 2 (Species A) = 0.8, Point 3 (Species B) = 1.1, Point 4 (Species B) = 1.4. The 3 nearest are Points 1, 2, and 3.

4

Majority Vote → Predict

Among the 3 neighbors: 2 are Species A, 1 is Species B. Majority = Species A. **Prediction: Species A.** If K=5 and neighbors were 3A + 2B, still Species A. If K=1, only Point 1 matters → Species A.

Key Insight: Changing K changes the decision boundary. K=1 creates complex, jagged boundaries (sensitive to noise). Large K creates smoother, simpler boundaries. The optimal K balances bias and variance for the specific dataset.

KNN Implementation: The Iris Dataset

The Iris dataset is a classic benchmark in machine learning, containing 150 samples of iris flowers across 3 species (Setosa, Versicolor, Virginica), with 4 features each: sepal length, sepal width, petal length, and petal width. It is ideal for demonstrating KNN classification.

Step 1: Load & Explore Data

Import the Iris dataset (e.g., via scikit-learn's `load_iris()`). Examine feature distributions, check for class balance (50 samples per species — perfectly balanced), and visualize pairwise feature relationships using scatter plots to observe natural clustering.

Step 2: Preprocess & Split

Normalize or standardize features (important for KNN since it uses distance). Split data into training (e.g., 80%) and test (20%) sets using `train_test_split`. Stratified splitting preserves class ratios in both sets.

Step 3: Train & Tune K

Fit KNN with different K values (e.g., K=1 to K=20). Use cross-validation to evaluate each K. Plot accuracy vs. K to identify the optimal value. For Iris, K=5 to K=7 typically yields the best test accuracy (~95–98%).

Step 4: Evaluate & Predict

Assess the final model using accuracy, confusion matrix, and classification report (precision, recall, F1-score). The confusion matrix reveals which species are confused with each other — typically Versicolor and Virginica are harder to distinguish than Setosa.